

Отчёт по лабораторной работе №9

Выполнила студентка НКАбд-02-25

Валерия Сергеевна Григорьева

Содержание

1	Цель работы	3
2	Порядок выполнения лабораторной работы	4
2.1	Реализация подпрограмм в NASM	4
2.2	Отладка программ с помощью GDB	8
3	Задание для самостоятельной работы	26
4	Вывод	33

1 Цель работы

Цель данной работы - приобретение навыков написания программ с использованием подпрограмм, знакомство с методами отладки при помощи GDB и его основными возможностями.

2 Порядок выполнения лабораторной работы

2.1 Реализация подпрограмм в NASM

Для начала работы я создала каталог для лабораторной работы №9, перешла в него и создала файл lab9-1.asm (Рисунок 2.1).

```
vsgrigoreva@fedora:~$ mkdir ~/work/arch--pc/lab09
vsgrigoreva@fedora:~$ cd ~/work/arch--pc/lab09
vsgrigoreva@fedora:~/work/arch--pc/lab09$ touch lab09-1.asm
```

Рисунок 2.1: Создание каталога и файла для работы

Затем я внимательно изучила текст листинга 9.1 и ввела его в файл lab8-1.asm. В данной программе вычисляется арифметическое выражение $f(x) = 2x + 7$ с помощью подпрограммы `_calcul`. Далее создала исполняемый файл и запустила его (Рисунок 2.2). Для корректной работы программы скопировала файл `in_out.asm` в каталог с файлом `lab9-1.asm`.

```
vsgrigoreva@fedora:~/work/arch--pc/lab09$ nasm -f elf lab09-1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
vsgrigoreva@fedora:~/work/arch--pc/lab09$ ./lab09-1
Введите x: 5
2x+7=17
vsgrigoreva@fedora:~/work/arch--pc/lab09$
```

Рисунок 2.2: Создание исполняемого файла и запуск программы

Программа корректно выполнила свою работу и завершила ее. Затем я изменила текст программы, добавив подпрограмму `_subcalcul` в подпрограмму `_calcul`, для вычисления выражения $f(g(x))$, где $g(x) = 3x - 1$ (Рисунок 2.3).

```
%include 'in_out.asm'

SECTION .data
    msg: DB 'Введите x: ',0
    result: DB '2x+7=',0

SECTION .bss
    x: RESB 80
    res: RESB 80

SECTION .text
GLOBAL _start
_start:
;-----
; Основная программа
;-----

    mov eax, msg
    call sprint

    mov ecx, x
    mov edx, 80
    call sread

    mov eax, x
```

```

    call atoi

    call _calcul ; Вызов подпрограммы _calcul
    mov eax,result
    call sprint
    mov eax,[res]
    call iprintLF

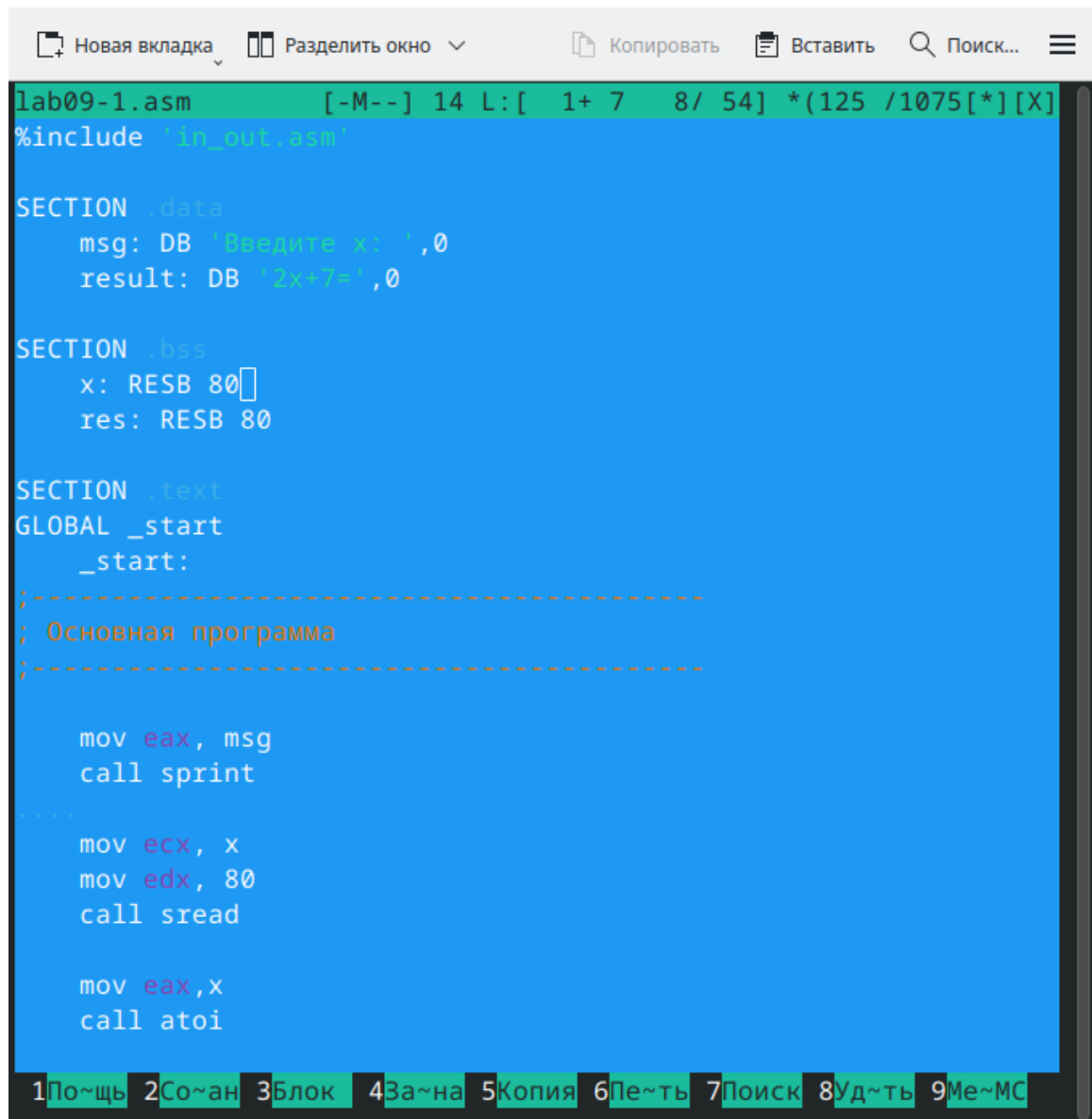
    call quit
;-----
; Подпрограмма вычисления
; выражения "f(g(x)) = 2x+7";

    _calcul:
    call _subcalcul
    mov ebx,2
    mul ebx
    add eax,7
    mov [res],eax
    ret ; выход из подпрограммы
;-----
; Подпрограмма вычисления
; выражения "g(x) = 3x-1"
;-----

    _subcalcul:
    mov ebx, 3
    mul ebx           ; eax = 3 * x
    sub eax, 1

```

ret



```
lab09-1.asm      [-M--] 14 L:[ 1+ 7 8/ 54] *(125 /1075[*][X]
#include 'in_out.asm'

SECTION .data
    msg: DB 'Введите x: ',0
    result: DB '2x+7=',0

SECTION .bss
    x: RESB 80
    res: RESB 80

SECTION .text
GLOBAL _start
_start:
;-----
; Основная программа
;-----

    mov eax, msg
    call sprint
    ....
    mov ecx, x
    mov edx, 80
    call sread

    mov eax, x
    call atoi
```

Рисунок 2.3: Измененный текст программы из файла lab9-1.asm

Далее я создала исполняемый файл и запустила его (Рисунок 2.4).

```
vsgrigoreva@fedora:~/work/arch--pc/lab09$ nasm -f elf lab09-1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
vsgrigoreva@fedora:~/work/arch--pc/lab09$ ./lab09-1
Введите x: 4
2x+7=29
vsgrigoreva@fedora:~/work/arch--pc/lab09$
```

Рисунок 2.4: Создание исполняемого файла и запуск программы

Программа корректно посчитала значение и завершила работу.

2.2 Отладка программ с помощью GDB

Для начала работы я создала файл lab9-2.asm в каталоге ~/work/arch-pc/lab08 (Рисунок 2.5).

```
vsgrigoreva@fedora:~/work/arch--pc/lab09$ touch lab09-2.asm
```

Рисунок 2.5: Создание файла lab9-2.asm

Затем я ввела в него текст программы из листинга 9.2 (Программа печати сообщения Hello world!) (Рисунок 2.6).


```
lab09-2.asm [-M--] 0 L: [ 1+ 0 1/ 25] *(0 [*][X]
SECTION .data
<----->msg1: db "Hello, ",0x0
<----->msg1Len: equ $ - msg1

<----->msg2: db "world!",0xa
<----->msg2Len: equ $ - msg2

SECTION .text
<----->global _start
_start:
    mov eax, 4
    mov ebx, 1
    mov ecx, msg1
    mov edx, msg1Len
    int 0x80

    mov eax, 4
    mov ebx, 1
    mov ecx, msg2
    mov edx, msg2Len
    int 0x80
```

Рисунок 2.6: Текст программы из файла lab9-2.asm

Далее я создала исполняемый файл, при этом добавив ключ „-g“ (Рисунок 2.7).

```

vsgrigoreva@fedora:~/work/arch--pc/lab09$ nasm -f elf -g -l lab09-2.lst lab09-2.asm
vsgrigoreva@fedora:~/work/arch--pc/lab09$ ld -m elf_i386 -o lab09-2 lab09-2.o
vsgrigoreva@fedora:~/work/arch--pc/lab09$ gdb lab09-2
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-2...
(gdb) 

```

Рисунок 2.7: Создание исполняемого файла и запуск отладчика

Далее я проверила работу программы, запустив ее в оболочке GDB с помощью команды `run` (Рисунок 2.8).

```

(gdb) run
Starting program: /home/vsgrigoreva/work/arch--pc/lab09/lab09-2

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Hello, world!
[Inferior 1 (process 4884) exited normally]
(gdb) 

```

Рисунок 2.8: Проверка работы программы в GDB

Для более подробного анализа программы установив брейкпоинт на метку `_start` и запустила её (Рисунок 2.9).

```
(gdb) break _start
Breakpoint 1 at 0x8048080: file lab09-2.asm, line 11.
(gdb) run
Starting program: /home/vsgrigoreva/work/arch--pc/lab09/lab09-2

Breakpoint 1, _start () at lab09-2.asm:11
11      mov eax, 4
(gdb) □
```

Рисунок 2.9: Установка брейкпоинта на метку `_start`

С помощью команды `disassemble` я посмотрела дисассимилированный код программы (Рисунок 2.10).

```
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08048080 <+0>:      mov     $0x4,%eax
    0x08048085 <+5>:      mov     $0x1,%ebx
    0x0804808a <+10>:     mov     $0x8049000,%ecx
    0x0804808f <+15>:     mov     $0x8,%edx
    0x08048094 <+20>:     int     $0x80
    0x08048096 <+22>:     mov     $0x4,%eax
    0x0804809b <+27>:     mov     $0x1,%ebx
    0x080480a0 <+32>:     mov     $0x8049008,%ecx
    0x080480a5 <+37>:     mov     $0x7,%edx
    0x080480aa <+42>:     int     $0x80
    0x080480ac <+44>:     mov     $0x1,%eax
    0x080480b1 <+49>:     mov     $0x0,%ebx
    0x080480b6 <+54>:     int     $0x80
End of assembler dump.
(gdb) █
```

Рисунок 2.10: Дисассимилированный код программы

Затем переключилась на отображение команд с Intel'овским синтаксисом (Рисунок 2.11).

```

(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08048080 <+0>:      mov     eax,0x4
    0x08048085 <+5>:      mov     ebx,0x1
    0x0804808a <+10>:     mov     ecx,0x8049000
    0x0804808f <+15>:     mov     edx,0x8
    0x08048094 <+20>:     int     0x80
    0x08048096 <+22>:     mov     eax,0x4
    0x0804809b <+27>:     mov     ebx,0x1
    0x080480a0 <+32>:     mov     ecx,0x8049008
    0x080480a5 <+37>:     mov     edx,0x7
    0x080480aa <+42>:     int     0x80
    0x080480ac <+44>:     mov     eax,0x1
    0x080480b1 <+49>:     mov     ebx,0x0
    0x080480b6 <+54>:     int     0x80
End of assembler dump.
(gdb) █

```

Рисунок 2.11: Переключение на отображение команд с Intel'овским синтаксисом

Различия отображения синтаксиса машинных команд в режимах АТТ и Intel. В Intel-синтаксисе операнды идут в порядке dst, src (например, `mov eax, 0x4`), регистры пишутся без %. В АТТ-синтаксисе обратный порядок src, dst (`mov $0x4, %eax`), используются префиксы % для регистров и \$ для констант.

Далее я включила режим псевдографики для более удобного анализа программы (Рисунок 2.12).

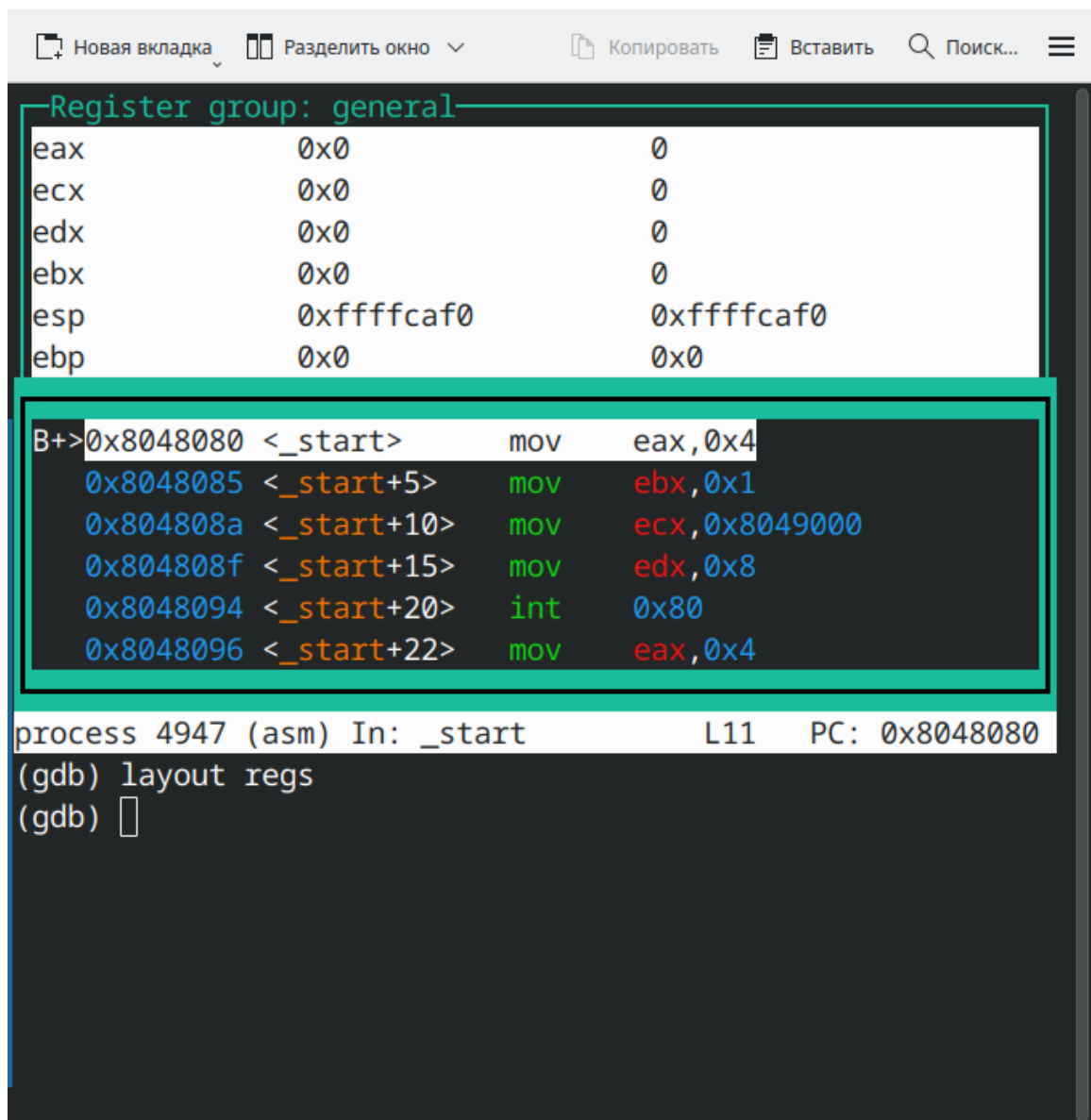


Рисунок 2.12: Режим псевдографики

2.2.1 Добавление точек останова

С помощью команды `info breakpoints` я проверила точку останова по имени метки (`_start`), установленную на предыдущих шагах (Рисунок 2.13).

```
(gdb) info breakpoints
Num      Type      Disp Enb Address      What
1        breakpoint keep y  0x08048080 lab09-2.asm:11
breakpoint already hit 1 time
(gdb) █
```

Рисунок 2.13: Точка останова

Точка действительно была установлена. Затем я установила еще одну точку останова по адресу инструкции, определив адрес инструкции `mov ebx,0x0` (Рисунок 2.14).

```
Новая вкладка  Разделить окно  Копировать  Вставить  Поиск...  ≡

Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcaf0  0xffffcaf0
ebp      0x0      0x0

0x804809b <_start+27>  mov     ebx,0x1
0x80480a0 <_start+32>  mov     ecx,0x8049008
0x80480a5 <_start+37>  mov     edx,0x7
0x80480aa <_start+42>  int     0x80
0x80480ac <_start+44>  mov     eax,0x1
b+ 0x80480b1 <_start+49>  mov     ebx,0x0

process 4947 (asm) In: _start      L11  PC: 0x8048080
(gdb) layout regs
(gdb) info breakpoints
Num      Type          Disp Enb Address      What
1        breakpoint    keep y  0x08048080  lab09-2.asm:11
        breakpoint already hit 1 time
(gdb) b *0x80480b1
Breakpoint 2 at 0x80480b1: file lab09-2.asm, line 24.
(gdb) □
```

Рисунок 2.14: Установка второй точки останова

Затем я еще раз посмотрела информацию о всех установленных точках останова (Рисунок 2.15). Точек останова теперь две.


```
(gdb) i b
Num      Type      Disp Enb Address      What
1        breakpoint keep y  0x08048080 lab09-2.asm:11
          breakpoint already hit 1 time
2        breakpoint keep y  0x080480b1 lab09-2.asm:24
(gdb) █
```

Рисунок 2.15: Информация о точке останова

2.2.2 Работа с данными программы в GDB

Я выполнила 5 инструкций с помощью команды `stepi` (или `si`) и проследила за изменением значений регистров (Рисунок 2.16). Значения регистров EAX, EBX, ECX и EDX изменяются.

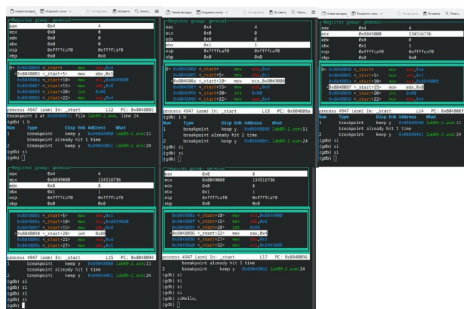


Рисунок 2.16: Работа команды `stepi`

Далее я посмотрела значение переменной `msg1` по имени (Рисунок 2.17).

```

Новая вкладка  Разделить окно  Копировать  Вставить  Поиск...
eax      0x4      4
eax      0x1      1
edx      0x7      7
edx      0x7      7
esp      0xffffcaf0  0xffffcaf0
ebp      0x0      0x0

0x804809b <_start+27>  mov     ebx,0x1
0x80480a5 <_start+37>  mov     edx,0x7
0x80480aa <_start+42>  int     0x80
0x80480ac <_start+44>  mov     eax,0x1
B+>0x80480b1 <_start+49>  mov     ebx,0x0
0x80480b6 <_start+54>  int     0x80
0x80480b8          add     BYTE PTR [eax],a1

process 4947 (asm) In: _start      L21      PC: 0x80480aa
(gdb) si                          4      b1
ds      Hello, 0x2b      43
es      0x2b      43
--Type <RET> for more, q to quit, c to continue without pag
ing--q
Quit
(gdb) x/1sb &msg1
0x8049000 <msg1>:      "Hello, "
(gdb) 

```

Рисунок 2.17: Значение переменной msg1

А затем значение msg2 по адресу (Рисунок 2.18).

```
Новая вкладка  Разделить окно  Копировать  Вставить  Поиск...  ≡

eax      0x4      4
eax      0x1      1
edx      0x7      7
edx      0x7      7
esp      0xffffcaf0  0xffffcaf0
ebp      0x0      0x0

0x804809b <_start+27>  mov     ebx,0x1
0x804808a <_start+10>  mov     ecx,0x8049000
0x804808f <_start+15>  mov     edx,0x8
0x8048094 <_start+20>  int     0x80
0x8048096 <_start+22>  mov     eax,0x4
0x804809b <_start+27>  mov     ebx,0x1
0x80480a0 <_start+32>  mov     ecx,0x8049008

process 4947 (asm) In: _start      L21      PC: 0x80480aa
(gdb) si                          4      b1
(gdb) x/1st 0x8049008
0x8049008 <msg2>:      01110111
0x8049009:      01101111
0x804900a:      01110010
0x804900b:      01101100
(gdb) x/1sb 0x8049008
0x8049008 <msg2>:      "world!\n\034"
(gdb) 
```

Рисунок 2.18: Значение переменной msg2

Далее я изменила первый символ переменной msg1 и msg2 (Рисунок 2.19).

```
Новая вкладка  Разделить окно  Копировать  Вставить  Поиск...  ≡

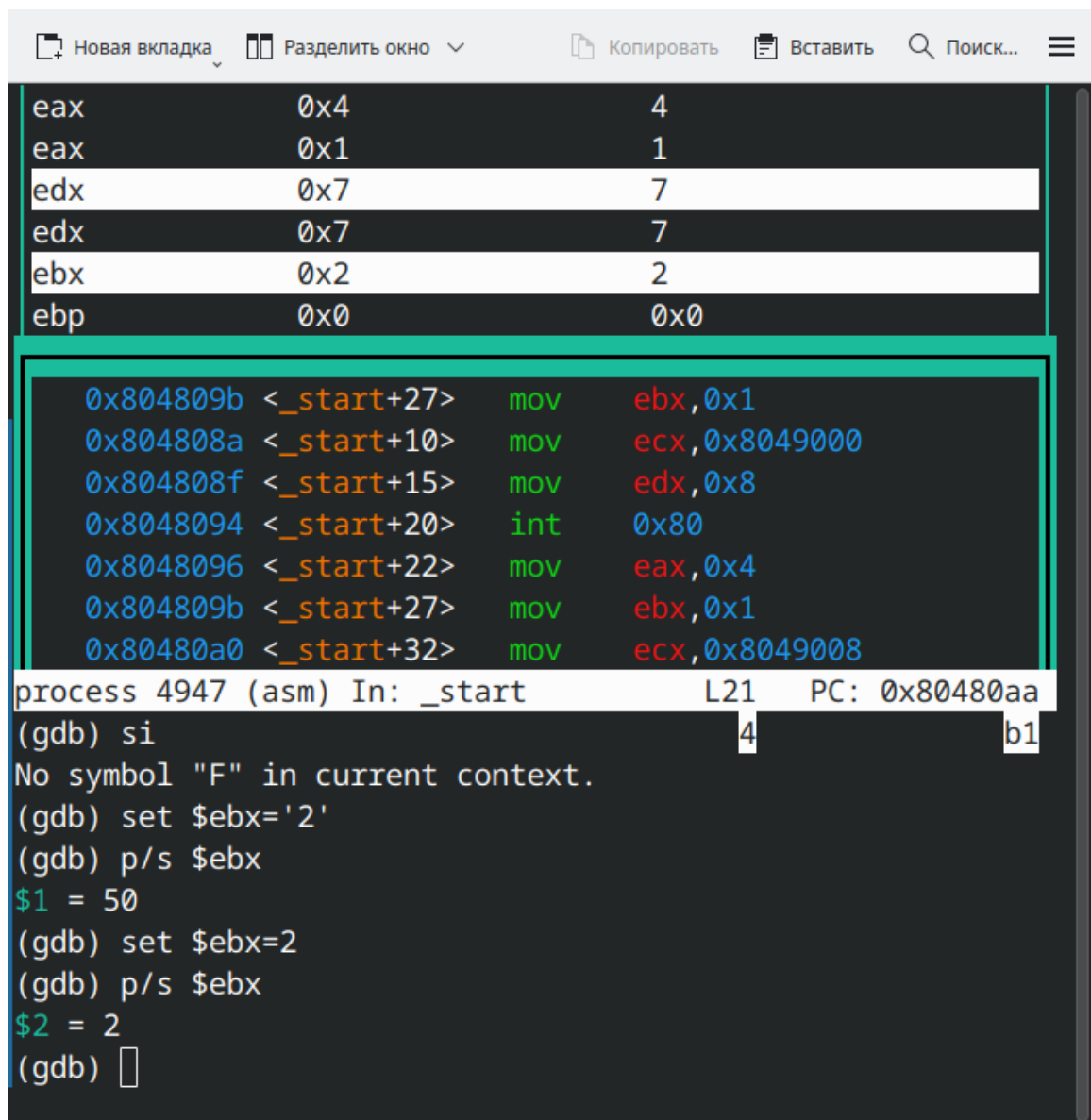
eax      0x4      4
eax      0x1      1
edx      0x7      7
edx      0x7      7
esp      0xffffcaf0 0xffffcaf0
ebp      0x0      0x0

0x804809b <_start+27>  mov     ebx,0x1
0x804808a <_start+10>  mov     ecx,0x8049000
0x804808f <_start+15>  mov     edx,0x8
0x8048094 <_start+20>  int     0x80
0x8048096 <_start+22>  mov     eax,0x4
0x804809b <_start+27>  mov     ebx,0x1
0x80480a0 <_start+32>  mov     ecx,0x8049008

process 4947 (asm) In: _start      L21      PC: 0x80480aa
(gdb) si                          4      b1
'msg1' has unknown type; cast it to its declared type
(gdb) set {char}&msg1='h'
(gdb) x/1sb &msg1
0x8049000 <msg1>:      "hello, "
(gdb) set {char}&msg2='q'
(gdb) x/1sb &msg2
0x8049008 <msg2>:      "qorld!\n\034"
(gdb) 
```

Рисунок 2.19: Изменение символов переменных

Затем я с помощью команды `set` изменила значение регистра `ebx` (Рисунок 2.20). Команда `p/s $ebx` выводит строку по адресу, хранящемуся в `ebx`, поэтому при `set $ebx=„2“` показывается ASCII-код символа, а при `set $ebx=2` — числовое значение.



```
Новая вкладка  Разделить окно  Копировать  Вставить  Поиск...  ≡

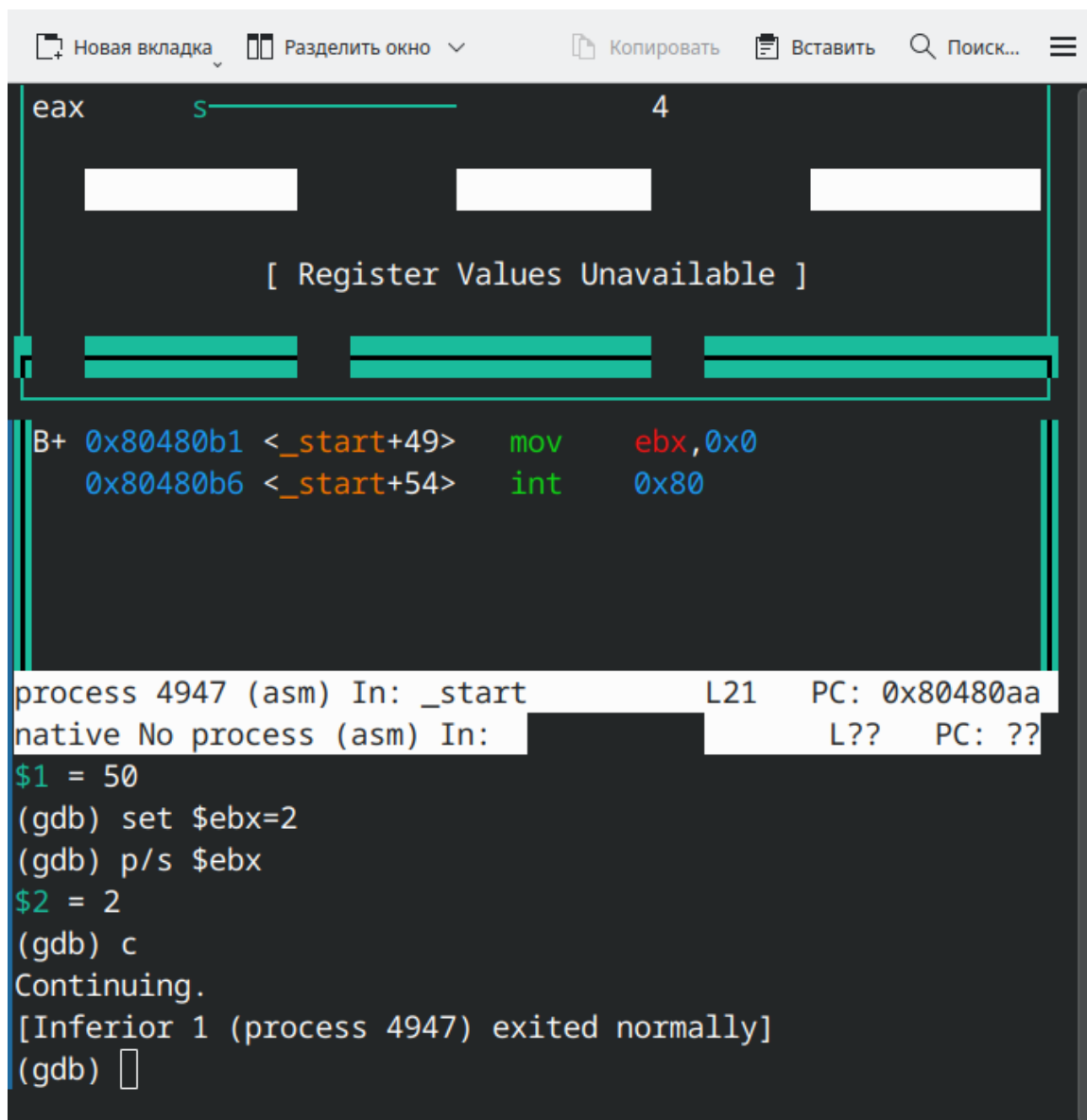
eax      0x4      4
eax      0x1      1
edx      0x7      7
edx      0x7      7
ebx      0x2      2
ebp      0x0      0x0

0x804809b <_start+27>  mov     ebx, 0x1
0x804808a <_start+10>  mov     ecx, 0x8049000
0x804808f <_start+15>  mov     edx, 0x8
0x8048094 <_start+20>  int     0x80
0x8048096 <_start+22>  mov     eax, 0x4
0x804809b <_start+27>  mov     ebx, 0x1
0x80480a0 <_start+32>  mov     ecx, 0x8049008

process 4947 (asm) In: _start      L21      PC: 0x80480aa
(gdb) si                          4      b1
No symbol "F" in current context.
(gdb) set $ebx='2'
(gdb) p/s $ebx
$1 = 50
(gdb) set $ebx=2
(gdb) p/s $ebx
$2 = 2
(gdb) □
```

Рисунок 2.20: Изменение значения регистра ebx

В конце я завершила работу с помощью программы continue и вышла из GDB с помощью команды quit (Рисунок 2.21).



The screenshot shows a GDB terminal window with a dark background. At the top, there is a menu bar with options in Russian: "Новая вкладка", "Разделить окно", "Копировать", "Вставить", "Поиск...", and a hamburger menu icon. Below the menu bar, the register values are displayed: "eax" with a green "S" and a value of "4". Below this, there are three white rectangular boxes. In the center, the text "[Register Values Unavailable]" is displayed. Below the boxes, there are three horizontal green bars. The assembly code is shown in the middle: "B+ 0x80480b1 <_start+49> mov ebx,0x0" and "0x80480b6 <_start+54> int 0x80". At the bottom, the GDB commands and their output are shown: "process 4947 (asm) In: _start L21 PC: 0x80480aa", "native No process (asm) In: L?? PC: ??", "\$1 = 50", "(gdb) set \$ebx=2", "(gdb) p/s \$ebx", "\$2 = 2", "(gdb) c", "Continuing.", "[Inferior 1 (process 4947) exited normally]", and "(gdb) " followed by a cursor.

```
eax S 4
[ Register Values Unavailable ]
B+ 0x80480b1 <_start+49> mov ebx,0x0
0x80480b6 <_start+54> int 0x80
process 4947 (asm) In: _start L21 PC: 0x80480aa
native No process (asm) In: L?? PC: ??
$1 = 50
(gdb) set $ebx=2
(gdb) p/s $ebx
$2 = 2
(gdb) c
Continuing.
[Inferior 1 (process 4947) exited normally]
(gdb) 
```

Рисунок 2.21: Завершение работы

2.2.3 Обработка аргументов командной строки в GDB

Я скопировала файл lab8-2.asm, созданный при выполнении лабораторной работы №8, с программой выводящей на экран аргументы командной строки в файл с именем lab09-3.asm (Рисунок 2.22).

```
vsgrigoreva@fedora:~/work/arch--pc/lab09$ cp ~/work/arch--pc/lab08/lab8-2.asm ~/work/arch--pc/lab09/lab09-3.asm
vsgrigoreva@fedora:~/work/arch--pc/lab09$
```

Рисунок 2.22: Копирование файла

Затем создала исполняемый файл и загрузила в отладчик, указав аргументы (Рисунок 2.23).

```
vsgrigoreva@fedora:~/work/arch--pc/lab09$ nasm -f elf -g -l lab09-3.lst lab09-3.asm
vsgrigoreva@fedora:~/work/arch--pc/lab09$ ld -m elf_i386 -o lab09-3 lab09-3.o
vsgrigoreva@fedora:~/work/arch--pc/lab09$ gdb --args lab09-3 a b 'c'
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-3...
(gdb) 
```

Рисунок 2.23: Загрузка в gdb

Далее установила точку останова перед первой инструкцией в программе и запустила ее (Рисунок 2.24).

```
(gdb) b _start
Breakpoint 1 at 0x8048148: file lab09-3.asm, line 8.
(gdb) run
Starting program: /home/vsgrigoreva/work/arch--pc/lab09/lab
09-3 a b c

This GDB supports auto-downloading debuginfo from the follo
wing URLs:
  <https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled
on' to .gdbinit.

Breakpoint 1, _start () at lab09-3.asm:8
8      pop ecx ; Извлекаем из стека в `ecx` количество
(gdb) □
```

Рисунок 2.24: Запуск первой инструкции

Затем я посмотрела позиции стека: вершину и остальные по адресу [esp+4] (Рисунок 2.25). Шаг изменения адреса равен 4, так как в этой программе каждый адрес в памяти занимает 4 байта, и стек хранит обычный список адресов, а чтобы перейти к следующему адресу, нужно сдвинуться ровно на 4 байта.


```
(gdb) x/x $esp
0xffffcae0:      0x00000004
(gdb) x/s *(void**)( $esp + 4)
0xffffcd2d:      "/home/vsgrigoreva/work/arch--pc/lab09/lab0
9-3"
(gdb) x/s *(void**)( $esp + 8)
0xffffcd5b:      "a"
(gdb) x/s *(void**)( $esp + 12)
0xffffcd5d:      "b"
(gdb) x/s *(void**)( $esp + 16)
0xffffcd5f:      "c"
(gdb) x/s *(void**)( $esp + 20)
0x0:      <error: Cannot access memory at address 0x0>
```

Рисунок 2.25: Позиции стека

3 Задание для самостоятельной работы

1. Преобразуйте программу из лабораторной работы №8 (Задание №1 для самостоятельной работы), реализовав вычисление значения функции () как подпрограмму (Рисунок 3.1).

```
%include 'in_out.asm'

SECTION .data
msg_func      db 'Функция: f(x)=6*x+13',0h
msg_res       db 'Результат: ',0h

SECTION .text
global _start
_start:

    pop ecx          ; ecx := argc
    pop edx          ; edx := argv0 (имя программы)
    sub ecx, 1        ; ecx := кол-во реальных аргументов (без имени программы)

    mov esi, 0        ; esi — накопитель суммы (32-bit)

    cmp ecx, 0
```

```

    jz .print_result

.loop_args:
    pop eax          ; eax := адрес очередного аргумента (строка)
    push ecx         ; сохранить ecx (atoi/подпрограммы могут изменить ecx)
    call atoi        ; преобразование строки (по адресу в eax) -> result в eax
                    ; теперь eax = числовое значение x
    pop ecx          ; восстановить ecx

    call _f          ; вычислить f(x), вход в eax, результат в eax

    add esi, eax     ; esi += f(x)

    loop .loop_args ; dec ecx; если !=0 -> переход к .loop_args

.print_result:
    mov eax, msg_func
    call sprintf

    mov eax, msg_res
    call sprintf

    mov eax, esi     ; результат в eax
    call iprintf     ; печать результата с переводом строки

    call quit

_f:

```

```
imul eax, 6      ; eax = eax * 6
add eax, 13      ; eax = eax + 13
ret
```

```
vsgrigoreva@fedora:~/work/arch--pc/lab09$ nasm -f elf sam1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab09$ ld -m elf_i386 -o sam1 sam1.o
vsgrigoreva@fedora:~/work/arch--pc/lab09$ ./sam1
Функция: f(x)=6*x+13
Результат: 0
vsgrigoreva@fedora:~/work/arch--pc/lab09$ ./sam1 7
Функция: f(x)=6*x+13
Результат: 55
vsgrigoreva@fedora:~/work/arch--pc/lab09$ █
```

Рисунок 3.1: Результат вывода программы

2. В листинге 9.3 приведена программа вычисления выражения $(3 + 2) \cdot 4 + 5$. При запуске данная программа дает неверный результат. Проверьте это. С помощью отладчика GDB, анализируя изменения значений регистров, определите ошибку и исправьте ее.

Я ввела текст программы из листинга 9.3 в файл `sam2.asm`, создала исполняемый файл и запустила `gdb` (Рисунок 3.2).

```

vsgrigoreva@fedora:~/work/arch--pc/lab09$ nasm -f elf -g -l sam2.lst sam2.asm
vsgrigoreva@fedora:~/work/arch--pc/lab09$ ld -m elf_i386 -o sam2 sam2.o
vsgrigoreva@fedora:~/work/arch--pc/lab09$ gdb --args sam2
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>

This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from sam2...

```

Рисунок 3.2: Запуск gdb

Затем я установила брейкпоинт на метку `_start` и запустила программу (Рисунок 3.3).

```

(gdb) b _start
Breakpoint 1 at 0x8048168: file sam2.asm, line 12.
(gdb) run
Starting program: /home/vsgrigoreva/work/arch--pc/lab09/sam2

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.

Breakpoint 1, _start () at sam2.asm:12
12      mov ebx,3

```

Рисунок 3.3: Запуск программы в gdb

Далее с помощью команды `stepi` проследила за изменением значений регистров (Рисунок 3.4).

```

(gdb) si
13      mov eax,2
(gdb) info registers eax ebx ecx edx
eax      0x0      0
ebx      0x3      3
ecx      0x0      0
edx      0x0      0
(gdb) si
14      add ebx,eax
(gdb) info registers eax ebx ecx edx
eax      0x2      2
ebx      0x3      3
ecx      0x0      0
edx      0x0      0
(gdb) si
15      mov ecx,4
(gdb) info registers eax ebx ecx edx
eax      0x2      2
ebx      0x5      5
ecx      0x0      0
edx      0x0      0
(gdb) si
16      mul ecx
(gdb) info registers eax ebx ecx edx
eax      0x2      2
ebx      0x5      5
ecx      0x4      4
edx      0x0      0
(gdb) si
17      add ebx,5
(gdb) info registers eax ebx ecx edx
eax      0x8      8
ebx      0x5      5
ecx      0x4      4
edx      0x0      0
(gdb) si
18      mov edi,ebx
(gdb) info registers eax ebx ecx edx
eax      0x8      8
ebx      0xa      10
ecx      0x4      4
edx      0x0      0
(gdb)

```

Рисунок 3.4: Отладка программы

Я определила ошибку и исправила ее.

```

#include 'in_out.asm'

SECTION .data
div: DB 'Результат: ',0

```

```

SECTION .text
GLOBAL _start
_start:

; ---- Вычисление выражения (3+2)*4+5

    mov ebx, 3
    mov eax, 2
    add ebx, eax
    mov ecx, 4
    mov eax, ebx
    mul ecx
    add eax, 5
    mov edi, eax

; ---- Вывод результата на экран
    mov eax, div
    call sprint
    mov eax, edi
    call iprintLF
    call quit

```

Затем создала исполняемый файл и запустила его (Рисунок 3.5).

```

vsgrigoreva@fedora:~/work/arch--pc/lab09$ nasm -f elf sam2.asm
vsgrigoreva@fedora:~/work/arch--pc/lab09$ ld -m elf_i386 -o sam2 sam2.o
vsgrigoreva@fedora:~/work/arch--pc/lab09$ ./sam2
Результат: 25
vsgrigoreva@fedora:~/work/arch--pc/lab09$ 

```

Рисунок 3.5: Создание исполняемого файла и запуск программы

Исправленная программа корректно посчитала значение выражения.

4 Вывод

В ходе выполнения работы были приобретены навыки написания программ с использованием подпрограмм, я познакомилась с методами отладки при помощи GDB и его основными возможностями.